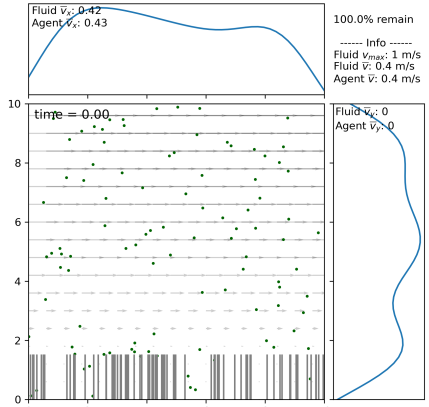
# USING PLANKTOS

## Agent-based transport and behavior in fluid environments

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Planktos




Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

Planktos documentation image `docs/_static/basic_ex_2d_agents.png` (commit `a1c2128`, GPLv3).

# WHAT YOU WILL BE ABLE TO BUILD

## Core workflow

- create a 2D or 3D `Environment`;
- specify outer-domain boundary conditions;
- generate or import a velocity field and mesh;
- initialize one or more vectorized `Swarms`;
- advance, inspect, plot, animate, and export.

## Research extensions

- encode deterministic, stochastic, or social behavior;
- choose sliding, sticky, or disabled mesh collisions;
- compute KDE and finite-time Lyapunov exponent fields;
- reproduce a current Vicsek-model example.

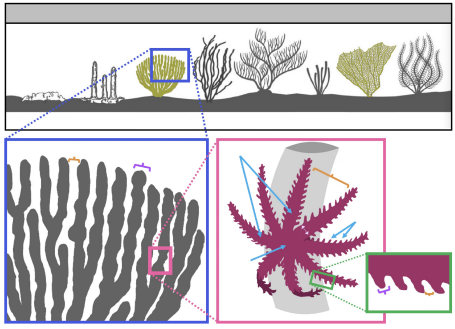
> Every code fragment targets **Planktos v1.0.3**, repository commit `a1c2128` (2026-08-19). Unless a slide explicitly continues prior state, treat each fragment as a separate pattern in a fresh environment.

# MODELING SCOPE: AGENTS RESPOND TO A PRESCRIBED FLOW

Planktos is designed for point-particle organisms or particles whose feedback on the surrounding flow is negligible. A useful template is

$$d\boldsymbol{X} = \big[\boldsymbol{u}(\boldsymbol{X}, t) + \boldsymbol{b}(\boldsymbol{X}, t; \theta)\big]\, dt + \boldsymbol{\sigma}\, d\boldsymbol{W}_t,$$

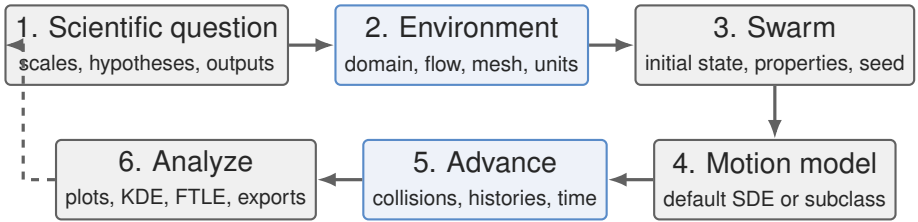
- $\boldsymbol{u}$: prescribed CFD or analytical velocity field;
- $\boldsymbol{b}$: swimming, settling, taxis, or social drift;
- $\boldsymbol{\sigma}$: stochastic variability, constant in space (it may still differ per agent);
- boundaries and immersed meshes constrain the proposed step.



Biological scales and gorgonian morphology: Hamlet et al. (2023), Fig. 1 (adapted from Kupfner Johnson & Hallock, 2020), CC BY 4.0.

A spatially varying $\boldsymbol{\sigma}(\boldsymbol{X})$ needs a Milstein-type scheme; the built-in Euler–Maruyama step is only order 0.5 in that case and does not support it directly. **One-way coupling.** Agent motion does *not* alter $\boldsymbol{u}$; use a coupled solver when feedback matters.

# THE END-TO-END WORKFLOW




> Treat plotting and diagnostics as part of the simulation loop: inspect early, then scale up.

Schematic: instructor's course notes.

# INSTALL FROM THE CLONED REPOSITORY

Clone the official repository, then make an isolated environment inside it.

```
git clone https://github.com/mountaindust/Planktos.git
cd Planktos
python -m venv .venv

# PowerShell
.\.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate

python -m pip install --upgrade pip
python -m pip install -e .
python -c "import planktos; print(planktos.__version__)"
```

Expected version for this deck: `1.0.3`.

## Editable install

`-e` points the environment at the clone, so a later `git pull` updates the import without reinstalling.

## Smoke test

Run `python examples/basic_ex_2d.py`; close each interactive plot to continue.

Adapted from the repository `README.md` Quickstart.

# DEPENDENCIES: WHAT IS INSTALLED, AND WHAT IS OPTIONAL

| Declared core | Role |
|---|---|
| Python $\geq 3.8$ | runtime (development currently uses newer Python) |
| NumPy $\geq 1.19$ | vectorized agent and field arrays |
| SciPy $\geq 1.10.1$ | interpolation, KDE, ODE integration |
| Matplotlib $\geq 3.0$ | plots and animations |
| pandas | per-agent properties |
| VTK/PyVista | $\geq 9.2$/ $\geq 0.44$; CFD/mesh I/O and VTK output |

Versions and extras are from `setup.cfg` at commit `a1c2128`.

## Optional extras

```
python -m pip install -e
    ".[STL,netCDF,test]"
```

- `numpy-stl`: STL input;
- `netCDF4`: NetCDF input;
- `pytest`: test suite.

## VTK installation

If binary resolution becomes difficult, the project documentation recommends a clean conda-forge/Miniforge environment.

# FFmpeg is needed only when saving animations

```
# Verify that the executable is on PATH
ffmpeg -version

# PowerShell can also locate it explicitly
Get-Command ffmpeg
```

**Without FFmpeg**

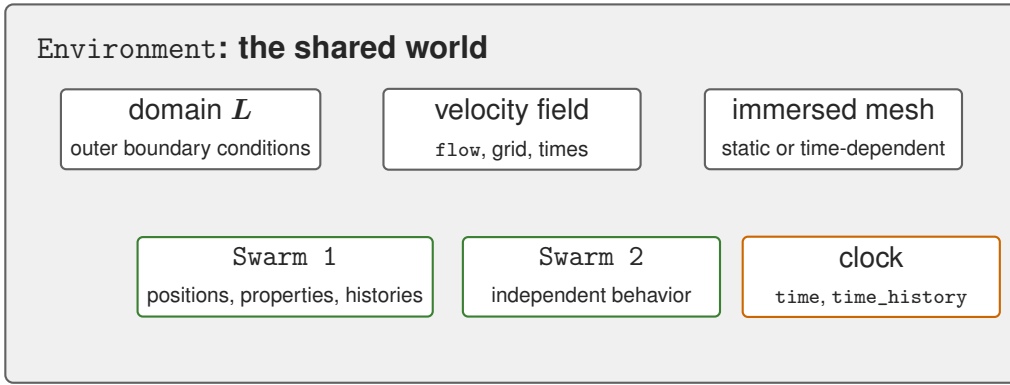
Simulations, static plots, and interactive animations still work. A filename passed to `plot_all` triggers the external movie writer.

**If it is not found**

1. install FFmpeg with the platform's package manager or official installer;
2. add its executable directory to `PATH`;
3. restart the terminal or IDE;
4. rerun `ffmpeg -version` in the same environment that runs Python.

> For long simulations, save every $k$-th state as a frame; rendering every time step is rarely useful.

# TWO OBJECTS ORGANIZE ALMOST EVERYTHING




- A swarm is a vectorized group: agents are rows of NumPy arrays, not separate Python objects.
- All swarms in one environment sample the same flow and mesh.

Schematic: instructor's course notes; object model from `planktos/_environment.py` and `_swarm.py`, commit `a1c2128`.

# CREATE THE FIRST ENVIRONMENT

```python
import planktos

env = planktos.Environment(
    Lx=20, Ly=5,
    x_bndry="zero",
    y_bndry=["noflux", "noflux"],
    rho=1000, mu=1.0,
    units="m")

print(env.L)        # [20, 5]
print(env.nu)       # mu/rho = 0.001
print(env.time)     # 0.0
```

## Defaults worth knowing

- $10 \times 10$ two-dimensional domain;
- zero velocity everywhere;
- `zero` boundaries in $x, y$ (`noflux` in $z$ when `Lz` is given);
- no swarms or immersed mesh;
- `time = 0.0`.

## Units are metadata

`units` does not rescale data automatically. Make the flow, mesh, agent parameters, gravity, and time convention consistent yourself.

# OUTER-DOMAIN BOUNDARY CONDITIONS

| Setting | What happens to an agent crossing the box | Typical use |
|---|---|---|
| `zero` | The agent is masked and no longer updated or plotted. | open/outflow boundary |
| `noflux` | The proposed displacement through the wall is projected along it. | solid outer wall |
| `periodic` | The agent exits one side and re-enters on the opposite side. | repeating domain |

Each axis accepts one string for both sides or a two-entry sequence:

```
x_bndry=["zero","noflux"].
```

The order is low-side, high-side: left/right, bottom/top, and low/high $z$.

**Do not conflate two layers**

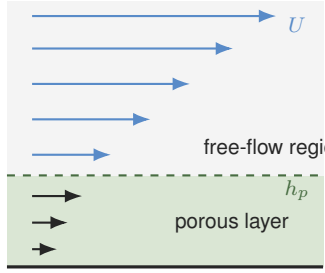
These settings govern the rectangular domain. Immersed-mesh collisions are controlled separately by `ib_condition` or `ib_collisions`.

# GENERATE A BRINKMAN-LAYER FLOW

```
env = planktos.Environment(
    Lx=20, Ly=5, rho=1000, mu=1000)

env.set_brinkman_flow(
    alpha=66, h_p=1.5,
    U=1, dpdx=1, res=101)

env.plot_flow()
```




Code adapted from `examples/basic_ex_2d.py`. Schematic: instructor's course notes.

- `alpha`: inverse hydraulic-permeability parameter (`alpha=0` is free flow);
- `h_p`: porous-layer height;
- `U`: velocity at the top of the domain;
- `dpdx`: pressure-gradient parameter;
- `res`: points across the resolved profile.

The profile is the analytic, steady, fully developed solution of Brinkman's equation (Brinkman 1949; Strickland et al. 2017), with a Darcy drag $-\alpha\mu\boldsymbol{u}$ in the porous layer per the docstring; the closed-form solution uses $e^{-\alpha h_p}$, so check the units of `alpha` (inverse length there) before reusing values. `mu` and `rho` must be set, or the call raises.

# READ THE FIELD REPRESENTATION BEFORE MOVING AGENTS

```
print(len(env.flow))
print([a.shape for a in env.flow])
print(env.flow_points)
print(env.flow_times)
print(env.L, env.units)
```

## Array convention

- static: `(nx, ny[, nz])`;
- time-dependent: `(nt, nx, ny[, nz])`;
- one array for each velocity component;
- `flow_points[i]` indexes spatial axis $i$.

## Interpolation contract

Planktos uses linear spatial interpolation on a rectilinear grid and cubic splines in time for time-dependent data.

## Preflight checks

- component shapes agree;
- coordinate arrays increase and match the axes;
- domain boundaries and mesh coordinates align;
- `flow_times` spans the intended run;
- speeds and units are physically plausible.

# SUPPLY A HAND-BUILT RECTILINEAR VELOCITY FIELD

```python
import numpy as np
import planktos

x = np.linspace(0, 2, 81)
y = np.linspace(0, 1, 41)
X, Y = np.meshgrid(x, y, indexing="ij")

u = 4*Y*(1-Y)          # streamwise component
v = np.zeros_like(u)

env = planktos.Environment(
    Lx=x[-1], Ly=y[-1], flow=[u, v])
env.flow_points = (x, y)
env.plot_flow()
```

## Why `indexing="ij"`?

Axis 0 is $x$, axis 1 is $y$. This matches the current Planktos storage convention and avoids a silent transpose.

## Time-dependent fields

Stack snapshots on axis 0 and pass `flow_times` to the constructor. The remaining axes stay $(x, y[, z])$.

## Coordinates

Set `flow_points` explicitly when the coordinates are not the constructor's uniform grid.

Convention cross-checked against current flow-generation and analysis tests.

# ADD A VECTORIZED SWARM

```
# Convenience method: automatically uses env
s = env.add_swarm(
    swarm_size=200,
    init="random", seed=2026,
    name="larvae", color="darkgreen")

# Equivalent explicit construction
s2 = planktos.Swarm(
    swarm_size=50, envir=env,
    init=(0.1, 0.5), seed=7)

print(s.N, s.positions.shape)
print(len(env.swarms))
```

**State layout**

- `positions`: masked $N \times d$ array;
- `velocities`: $N \times d$;
- `accelerations`: $N \times d$;
- `props`: per-agent DataFrame;
- `shared_props`: shared dictionary.

> Vectorization is the design choice: write array operations over agents whenever possible.

# CHOOSE INITIAL POSITIONS DELIBERATELY

```python
# Uniform random over the domain
s_random = env.add_swarm(500, init="random", seed=1)

# One shared starting point
s_point = env.add_swarm(100, init=(0.15, 0.50), seed=2)

# Explicit N-by-d array
rng = np.random.default_rng(3)
xy = np.column_stack((
    rng.uniform(0.05, 0.15, 300),
    rng.uniform(0.20, 0.80, 300)))
s_patch = env.add_swarm(init=xy, seed=3)

# Grid, optionally excluding closed mesh interiors
s_grid = env.add_swarm(
    init="grid", grid_dim=(80, 40), testdir="x1")
```

## Interpretation

- `random`: a uniform domain sample;
- point: a release cohort;
- $N \times d$ array: measured or designed initial condition (its row count sets $N$; `swarm_size` is ignored);
- `grid`: transport-map or FTLE seed lattice.

## Grid masking

`testdir` is meaningful only when a closed immersed mesh is loaded and the chosen ray direction gives a robust inside/outside test.

# SHARED PROPERTIES, INDIVIDUAL VARIATION, AND SEEDS

```python
import numpy as np

# Same for every agent
s.shared_props["mu"] = np.array([0.0, 0.0])
s.shared_props["cov"] = np.eye(2)*(0.01**2)
s.shared_props["diam"] = 1.0e-3

# One value per agent
rng = np.random.default_rng(2026)
s.props["swim_speed"] = rng.lognormal(
    mean=np.log(0.02), sigma=0.15, size=s.N)

# Uniform access: scalar/array as appropriate
speed = s.get_prop("swim_speed")
diam = s.get_prop("diam")
```

## The two random streams

- `seed=` initializes `s.rndState`, used by built-in stochastic motion.
- A separate NumPy generator should seed any user-created initial conditions or properties.

## Covariance units

The default SDE uses step covariance `cov` $dt$. Thus `cov` has units of length$^2$/time, not length$^2$ per step.

# THE DEFAULT MODEL IS EULER–MARUYAMA DRIFT–DIFFUSION

For each active agent, the default `apply_agent_model` approximates

$$d\boldsymbol{X} = [\boldsymbol{u}(\boldsymbol{X}, t) + \boldsymbol{\mu}]\, dt + \boldsymbol{\sigma}\, d\boldsymbol{W}_t, \qquad \texttt{cov} = \boldsymbol{\sigma}\boldsymbol{\sigma}^T = 2\boldsymbol{D}.$$

One step is

$$\boldsymbol{X}^{n+1,*} = \boldsymbol{X}^n + [\boldsymbol{u}(\boldsymbol{X}^n, t^n) + \boldsymbol{\mu}]\, \Delta t + \mathcal{N}(\boldsymbol{0}, \texttt{cov}\, \Delta t)\,,$$

followed by outer-boundary and immersed-mesh handling.

**Deterministic tracer**

For spatial dimension $d$, set `mu = np.zeros(d)` and `cov = np.zeros((d,d))`; then motion is forward Euler using the local fluid velocity.

**Time-step audit**

Check both temporal convergence and collision resolution. A step can be statistically reasonable yet too large to resolve a narrow obstacle.

Adapted from the `Swarm.apply_agent_model` and `motion.Euler_brownian_motion` docstrings (commit `a1c2128`); scheme: Kloeden & Platen (1992), §10.2. A `'D'` property may replace `'cov'`; it is converted as $\texttt{cov} = 2D$.

# ADVANCE THE SWARM AND READ ITS HISTORIES

```python
import numpy as np

dt = 0.1
for _ in range(round(24.0/dt)):
    s.move(dt, silent=True)

print(env.time)
print(len(env.time_history))
print(len(s.full_pos_history))

# Current diagnostics
mask = np.ma.getmaskarray(s.positions)
active = ~np.all(mask, axis=1)
print(active.sum(), "agents remain active")
print(s.velocities.mean(axis=0))
```

## What `move` manages

1. calls `apply_agent_model(dt)`;
2. stores prior position/velocity/property state;
3. estimates velocity and acceleration;
4. applies boundaries and mesh collisions;
5. calls `after_move(dt)`;
6. advances environment time.

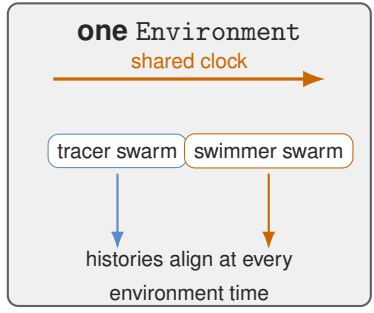
History note: `full_pos_history` and `full_vel_history` return stored history plus the current state without mutating it.

# MOVE MULTIPLE SWARMS ON ONE SHARED CLOCK

```
tracers = env.add_swarm(
    500, seed=10, name="tracers", color="navy")
swimmers = env.add_swarm(
    500, seed=11, name="swimmers", color="orange")
swimmers.shared_props["mu"] = np.array([0.02, 0.0])

for _ in range(400):
    env.move_swarms(
        dt=0.05,
        ib_collisions="sliding",
        silent=True)
```

Default differs: `move_swarms` uses `ib_collisions="sliding"` unless told otherwise, overriding each swarm's `ib_condition`; pass `ib_collisions="default"` to honor each swarm's own setting.




**Avoid double time advancement**

Do not call `s1.move(dt)` and then `s2.move(dt)` when both should occupy the same time step. Use `env.move_swarms`.

Schematic: instructor's course notes.

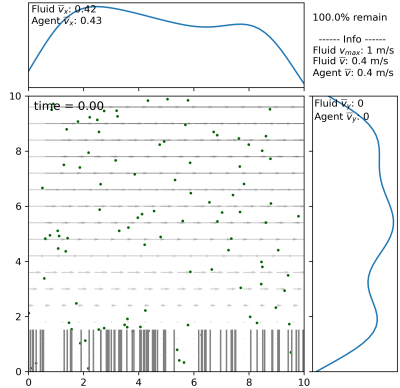
# MAKE AN EARLY DIAGNOSTIC PLOT

```python
# Current state with velocity arrows
s.plot(
    fluid="quiver",
    dist="density",
    circ_rad=0.08,
    plot_heading=False)

# Closest stored state to t=8.0
s.plot(
    t=8.0,
    filename="snapshot.png",
    fluid="vort",
    save_kwargs={"dpi": 200})
```

## 2D fluid overlays

`fluid="quiver"` shows velocity vectors; `fluid="vort"` shows scalar vorticity. These overlays are 2D-only.



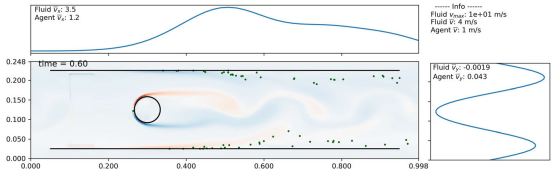

Not the output of the calls above, but a documentation render of the same example: Planktos documentation image `docs/_static/basic_ex_2d_agents.png` (commit `a1c2128`, GPLv3).

# KDE TURNS PARTICLES INTO A DISTRIBUTION DIAGNOSTIC

```
# Gaussian KDE with Scott's factor
s.plot(dist="density", fluid="vort")

# Covariance-based width
s.plot(dist="cov", fluid="vort")

# User bandwidth factor or a histogram
s.plot(dist=0.35)
s.plot(dist="hist")
```




IB2d cylinder flow, agents, vorticity, and marginal density: Strickland et al. (2022), Fig. 4, *Bull. Math. Biol.* 84:72; educational reproduction (article not openly licensed).

- Compare concentration shape, tails, and obstacle wakes.
- Report the bandwidth rule with any quantitative comparison.
- Check sensitivity to sample size and boundary truncation.
- A KDE is a smoothed estimator, not a separately solved concentration field.

# ANIMATE SELECTIVELY; EXPORT ANALYSIS-READY DATA

```python
# Use every fifth stored state
frames = range(0, len(s.full_pos_history), 5)
s.plot_all(
    movie_filename="trial_01.mp4",
    frames=frames, fps=10,
    fluid="vort", plot_heading=False)

# Data products
s.save_data("results", "trial_01")
s.save_pos_to_csv("results/trial_01_positions.csv")
s.save_pos_to_vtk("results", "trial_01")
```

What `save_data` writes: CSV files for the position history (with mask and time stamps) and for the current velocities and accelerations; `props` to JSON; `shared_props` to .npz. `props_history` is not saved. `save_pos_to_vtk` writes one PolyData file per stored time.

Calls match current `Swarm.plot_all` and save-method signatures.

## Choose the product by use

- `save_data`: position history plus current kinematics and properties (file formats at left);
- CSV: portable position history;
- VTK: ParaView/VisIt workflow;
- MP4: communication, not raw data.

## Cost control

Use `frames` for temporal subsampling and `downsamp` for display-only agent subsampling. Statistics still use the full population.

# CUSTOM BEHAVIOR: OVERRIDE ONE HOOK, NOT THE HARNESS

```python
class MySwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        # Compute proposed N-by-d positions.
        new_positions = ...
        return new_positions
```

## Contract

- keep the signature `(self, dt)`;
- return an $N \times d$ array;
- do not assign `positions`, `velocities`, or `accelerations`;
- let `move` record history and apply collisions.

Current contract: `planktos/_swarm.py` and `examples/ex_agent_behavior.py`.

## Current versus historical API

The 2022 paper describes a historical override named `get_positions`. The current v1.0.3 repository and examples use `apply_agent_model(self, dt)`.

> Use `after_move(dt)` for state changes based on the accepted, collision-corrected endpoint.

# EXAMPLE: FLUID ADVECTION PLUS DIRECTED SWIMMING

```python
import numpy as np
import planktos

class TargetSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        target = np.array([1.8, 0.5])
        direction = target - self.positions
        norm = np.linalg.norm(direction, axis=1)[:, None]
        direction = direction/np.maximum(norm, 1.0e-12)

        speed = self.get_prop("swim_speed")[:, None]
        velocity = self.get_fluid_drift() + speed*direction
        return self.positions + velocity*dt
```

Pattern adapted from `examples/ex_agent_behavior.py`.

## Useful field queries

- `get_fluid_drift()`;
- `get_dudt()`;
- `get_fluid_mag_gradient()`;
- `get_DuDt()`.

## Numerical hygiene

Guard normalizations, preserve masks, and call expensive interpolants once per time step rather than once per agent.

# DETERMINISTIC ODE MOTION WITH THE RK45 WRAPPER

```python
import planktos
from planktos import motion

class AccurateTracer(planktos.Swarm):
    def apply_agent_model(self, dt):
        ode = motion.tracer_particles(
            self, incl_dvdt=False)
        t0 = self.envir.time
        # positions is a masked array; strip masked
        # (exited) rows before RK45, as calculate_FTLE does
        return motion.RK45(
            ode, t0, self.positions.copy(), t0 + dt,
            rtol=1.0e-6, atol=1.0e-9)
```

About the comment: for brevity the listing passes the whole masked array. SciPy drops the mask, so exited rows are integrated from stale coordinates. Strip them as `calculate_FTLE` does, `y = pos[~mask[:,0]]`, and scatter the result back. (`incl_dvdt=False` makes the tracer ODE act on $N \times d$ positions only.)

## Current solver contract

`motion.RK45` wraps SciPy `solve_ivp`; the right-hand side has signature `fun(t, x)` and returns the same 2D shape as `x`.

## Two time scales

The internal ODE solver adapts substeps, but Planktos applies boundary and mesh collision handling only after the outer `move(dt)` proposal.

# COMBINE AN ODE DRIFT WITH STOCHASTIC MOTION

```python
import numpy as np
import planktos
from planktos import motion

def social_bias(pos):
    return 0.1*(pos.mean(axis=0) - pos)

def behavior_ode(swarm):
    def rhs(t, x):
        N = x.shape[0]//2
        vel = swarm.get_fluid_drift(t, x[:N])
        vel += social_bias(x[:N])
        return np.concatenate((
            vel, np.zeros((N, x.shape[1]))))
    return rhs

class SocialSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        return motion.Euler_brownian_motion(
            self, dt, ode=behavior_ode(self))
```

Adapted from `examples/ex_ode_gen.py`.

## State convention

An ODE passed to `Euler_brownian_motion` receives $2N \times d$: positions followed by velocities. It returns their time derivatives in the same layout.

## Fluid drift is explicit

When an `ode` is supplied, include fluid interaction in that ODE if it is part of the model; it is not added automatically there.

The swarm's `mu` property is still added to the ODE's velocity rows (zero by default), and the step is the Taylor update $\Delta \boldsymbol{x} = \boldsymbol{v}\,\Delta t + \frac{1}{2}\boldsymbol{a}\,\Delta t^2$ plus the `cov` noise.

# BUILT-IN INERTIAL-PARTICLE DYNAMICS (1/2)

**Set $\rho$, $\mu$, $\nu$ in the constructor:** the constructor computes the third value correctly, but in v1.0.3 the `nu` and `rho` property setters compute $\mu = \nu/\rho$ instead of $\mu = \nu\rho$ when $\mu$ is unset (an upstream bug, still present on `master` as of October 2026; not yet filed as a GitHub issue).

```
import numpy as np
import planktos
from planktos import motion

zero = np.zeros((3, 3))
env = planktos.Environment(
    Lx=1, Ly=1, rho=1000, nu=1e-6,
    char_L=0.01, U=0.1,
    flow=[zero, zero.copy()])
```

The listing continues on the next slide with the Swarm subclass. Here $\mathrm{Re} = U\,$`char_L`$/\nu = 0.1 \cdot 0.01/10^{-6} = 1000$, and the constructor sets $\mu = \nu\rho = 10^{-3}$.

More on the setters: once all three are set, a setter only warns and leaves the other two unchanged. The bug is still present on `master` as of 2026-09-16; no public GitHub issue for it was found as of 2026-10-11.

## Required environment data

A velocity field (explicit zeros are valid), `char_L`, `U`, `nu`, and gravity; fluid density unless `R` is supplied.

## Required particle data

Density ratio `R` (or particle `rho`) and diameter `diam`.

## Model regime

The implementation is the linearized Maxey–Riley form documented for $R/\mathrm{St} \gg 1$; verify that approximation for the particles and flow under study. Here, the displayed parameters (with `R` and `diam` from slide 2/2) give $R/\mathrm{St} \approx 120$.

Setter behavior: `Environment.rho/mu/nu` in `planktos/_environment.py`, commit `a1c2128`.

# BUILT-IN INERTIAL-PARTICLE DYNAMICS (2/2)

```
class InertialSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        ode = motion.inertial_particles(self)
        # positions is a masked array; strip masked
        # (exited) rows before RK45, as calculate_FTLE does
        # NB: move() overwrites velocities with
        # (x_new-x_old)/dt; persist velocity in props
        # for a true second-order model
        state = np.ma.concatenate(
            (self.positions, self.velocities))
        out = motion.RK45(
            ode, self.envir.time, state,
            self.envir.time + dt)
        return out[:self.N]

s = InertialSwarm(envir=env, seed=4)
s.shared_props.update({"R": 2/3, "diam": 1e-4})
```

## The ODE being integrated

With state $(\boldsymbol{x}, \boldsymbol{v})$ stacked as $2N \times d$,

$$\dot{\boldsymbol{v}} = \tfrac{3R}{2}\tfrac{D\boldsymbol{u}}{Dt} - \tfrac{R}{\mathrm{St}}(\boldsymbol{v} - \boldsymbol{u}) + \big(1 - \tfrac{3R}{2}\big)\boldsymbol{g},$$

$$R = \frac{2\rho_f}{\rho_f + 2\rho_p}, \; \mathrm{St} = \tfrac{2}{9}\big(\tfrac{a}{L}\big)^2 \mathrm{Re},$$

$a = \texttt{diam}/2$.

Check:
$\mathrm{St} = \frac{2}{9}(5 \times 10^{-3})^2 \cdot 1000 \approx 5.6 \times 10^{-3}$, so $R/\mathrm{St} \approx 0.667/0.0056 \approx 120$. $R = 2/3$ is neutrally buoyant, so the gravity term vanishes.

Model: `motion.inertial_particles` docstring, after Maxey & Riley (1983) and Haller & Sapsis (2008).

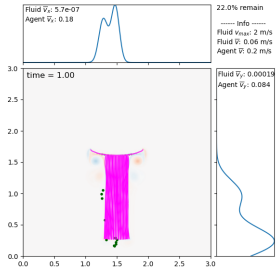
# IMMERSED-MESH COLLISION MODES

| Mode | Proposed step after a mesh hit |
|---|---|
| `sliding` | Project motion along the boundary, recursively using remaining displacement. |
| `sticky` | Stop at the first point of intersection for that time step. |
| `None` | Ignore immersed meshes. |
| `'default'` | Use the Swarm's `ib_condition` (default `'sliding'`). |

Set a swarm default with `ib_condition`; override per step with

```
s.move(dt, ib_collisions="sticky").
```

`s.ib_collision_idx` is a per-agent array of the last-step element index hit, $-1$ if none: precisely, the element of the *first* intersection during the most recent step (a sliding agent may go on to touch others). It is reset every step for agents still in the domain, so a captured agent held in place reads $-1$ on later steps; keep capture in `props`.




Sticky agents and a moving IB2d boundary; Planktos documentation image `docs/_static/ib2d_mvbnd_sticky.png` (commit `a1c2128`, GPLv3).

## Geometry restriction

Avoid immersed structures that intersect periodic box boundaries; that interaction is not implemented in the current API.

# MAKE “CAPTURED” A PERSISTENT BIOLOGICAL STATE

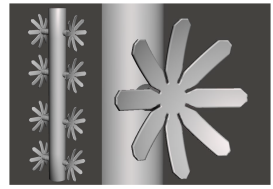
```python
import numpy as np
import planktos
from planktos import motion

class CaptureSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        proposed = motion.Euler_brownian_motion(self, dt)
        held = self.get_prop("captured")[:, None]
        return np.ma.where(held, self.positions, proposed)

    def after_move(self, dt):
        hit = self.ib_collision_idx >= 0
        self.props.loc[hit, "captured"] = True
        self.props.loc[hit, "color"] = "gold"

s = CaptureSwarm(
    envir=env, ib_condition="sticky",
    store_prop_history=True)
s.props["captured"] = np.zeros(s.N, dtype=bool)
s.props["color"] = np.full(s.N, s.shared_props["color"])
```

Pattern adapted from `examples/ex_sticky_seafan_3d.py`.



Cylinder and polyp geometry: Hamlet et al. (2023), Fig. 6a, CC BY 4.0.

## Why both hooks?

`after_move` records new contacts after collision correction. `apply_agent_model` prevents previously captured agents from leaving on later steps.

# IMPORT AN IB2D VELOCITY FIELD AND 2D MESH

```
env = planktos.Environment()

# From the repository root, first follow
# examples/ib2d_data/README.txt to download the data.
env.read_IB2d_fluid_data(
    "examples/ib2d_data",
    dt=5.0e-5, print_dump=1000)

env.read_IB2d_mesh_data(
    "examples/ib2d_data/channel.vertex",
    method="proximity")

env.plot_envir()
s = planktos.Swarm(
    envir=env, init=(0.1*env.L[0], 0.5*env.L[1]),
    seed=1)
```

## Time mapping

Successive printed flow snapshots are separated by

$$\Delta t_{\text{data}} = \texttt{dt} \times \texttt{print_dump} = 0.05.$$

## Mesh association

`proximity` connects nearby vertices. It can infer resolution from loaded flow or accept `res=` explicitly.

**Inspect connectivity.** Crossing segments without a shared vertex can create collision leaks. Always call `plot_envir()`.

Adapted from `examples/ex_ib2d_ibmesh.py`; the external dataset is linked from `examples/ib2d_data/README.txt`.

# IMPORT MATRIX: CFD FIELDS AND 3D GEOMETRY

| Source | Current call pattern | Audit item |
|---|---|---|
| IBAMR 3D VTK | `env.read_IBAMR3d_vtk_data(path, d_start=0, d_finish=None, vel_conv=None, grid_conv=None)` | dump range; spatial and velocity conversions |
| COMSOL VTU | `env.read_comsol_vtu_data(file, vel_conv=None, grid_conv=None)` | cell-centered versus nodal grid; units |
| NetCDF | `env.load_NetCDF(file)` then `env.read_NetCDF_flow("u","v", time_name="time", dim_reorder=...)` | variable names, dimension order, component axis, calendar/time units |
| STL mesh | `env.read_stl_mesh_data(file, unit_conv=None)` | mesh and flow share the same coordinate frame |
| IB2d mesh | `env.read_IB2d_mesh_data(file, method="adjacent"\|"proximity"\|"hull")` | breaks/additional edges; closed geometry |

> The loader may reset `env.L` to match the data. Read back `L`, `flow_points`, `flow_times`, `fluid_domain_LLC`, and units after every import.

Signatures audited against `planktos/_environment.py`, v1.0.3.

# ALIGN FLOW, MESH, AND PHYSICAL UNITS BEFORE RELEASE

```python
# For appropriate cell-centered COMSOL data,
# regrid immediately after the flow import.
# Match these flags to the data's actual BCs;
# this example treats all axes as nonperiodic.
env.center_cell_regrid(
    periodic_dim=(False, False, False))

# Only then load the immersed geometry.
env.read_stl_mesh_data("geometry.stl")

print("domain:", env.L, env.units)
print("flow origin:", env.fluid_domain_LLC)
print("flow axes:", [p[[0, -1]] for p in env.flow_points])
print("mesh shape:", None if env.ibmesh is None
      else env.ibmesh.shape)

env.plot_envir()
env.plot_flow()
```

## One consistent conversion ledger

Record conversions for length, velocity, time, density, viscosity, gravity, particle diameter, and mesh coordinates. Prefer converting on import with `vel_conv`, `grid_conv`, or `unit_conv`.

## Visual mesh audit

Look for crossing edges, missing closure, duplicated surfaces, mesh outside the flow domain, and unintended contact with a periodic face.

Pass the flags explicitly: in v1.0.3 the `center_cell_regrid` docstring states a default of `(True, True, False)`, but the signature default is `(False, False, False)`. The regrid does not shift meshes already loaded, hence “only then”.

# FTLE REVEALS FINITE-TIME STRETCHING OF A MOTION MODEL

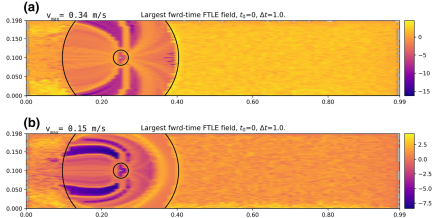
Let $\tau = T - t_0 > 0$. For the flow map $\mathbf{\Phi}_{t_0}^{T}$, the largest forward-time exponent is

$$\sigma_{t_0}^{\tau}(\boldsymbol{x}_0) = \frac{1}{\tau} \log \sqrt{\lambda_{\max}[(D\mathbf{\Phi})^T D\mathbf{\Phi}]}.$$

```
env.calculate_FTLE(
    grid_dim=(512, 128),
    t0=0, T=0.1, dt=0.001)
env.plot_2D_FTLE()

# Backward from physical time 0.1 to 0.0:
# current API uses T as the forward pseudo-time endpoint.
env.calculate_FTLE(
    (512, 128), t0=0.1, T=0.2,
    dt=0.001, backward=True)
```

Calls adapted from `examples/ex_produce_ftle_2d.py`.




Agent-based forward-time FTLE comparison: Strickland et al. (2022), Fig. 5, *Bull. Math. Biol.* 84:72; educational reproduction (article not openly licensed).

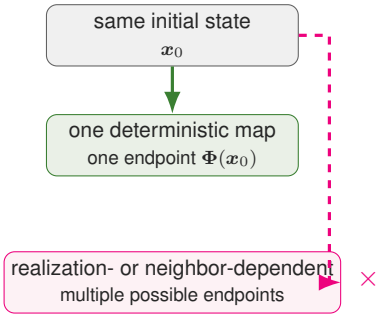
## Two API distinctions

- The current call requires `T > t0`; the elapsed interval is `T-t0` (`T` is an end time, although the docstring calls it "integration time"). With `backward=True`, physical time ends at `2*t0-T`; tracers only.
- `FTLE_smallest` is a forward-map contraction exponent, not backward-time FTLE.

# WHEN AN AGENT-BASED FTLE FIELD IS MEANINGFUL

The `swrm=` route is appropriate when the seeded agents define one reproducible finite-time map:

- the motion rule is deterministic;
- agents use the same rule and parameter set;
- each trajectory acts independently of the other seeded agents;
- the map is resolved well enough in time and on the initial-condition grid.




## Not an ordinary FTLE map

Stochastic realizations or social rules that depend on the seeded population do not define a single deterministic particle map. Use ensemble dispersion, encounter, or density diagnostics instead.

Qualification follows Strickland et al. (2022), Sections 2.1 and 4.2.

FTLE is a property of the chosen finite-time map, not of “complex-looking” trajectories in general.

Schematic: instructor’s course notes.

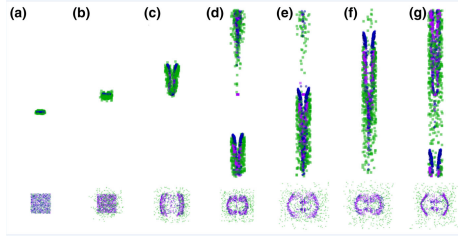
# CAPSTONE: VICSEK AGENTS AROUND AN IMMERSED CYLINDER

```python
import numpy as np
import planktos

env = planktos.Environment(
    Lx=1, Ly=0.25,
    x_bndry="periodic",
    y_bndry="periodic")
env.read_IB2d_mesh_data(
    "examples/vicsek_mesh/channel.vertex",
    method="proximity", res=0.001953125)

rng = np.random.default_rng(2026)
N = 500
IC = np.empty((N, 2))
IC[:, 0] = rng.uniform(0.075, 0.125, N)
IC[:, 1] = rng.uniform(0.050, 0.200, N)
```

Setup: current `examples/ex_vicsek_model_2d.py`; separate NumPy seed added.




Motivating 3D result: Strickland et al. (2022), Fig. 9, *Bull. Math. Biol.* 84:72; educational reproduction (article not openly licensed).

## Null versus flow experiment

The repository example runs without the large external flow dataset. Load matching IB2d flow to ask how alignment survives an unsteady wake.

# CAPSTONE: IMPLEMENT ALIGNMENT WITH PERIODIC DISTANCES

```
class Vicsek(planktos.Swarm):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.shared_props.update(v=0.02, nu=0.5, r=0.1)
        angle = self.get_prop("nu") \
              * (self.rndState.random(self.N)-0.5)
        self.velocities += self.get_prop("v") \
              * np.column_stack((np.cos(angle), np.sin(angle)))

    def apply_agent_model(self, dt):
        theta = np.empty(self.N)
        L = np.asarray(self.envir.L)
        for i, origin in enumerate(self.positions):
            delta = (self.positions-origin + L/2) % L - L/2
            near = np.linalg.norm(delta, axis=1) < self.get_prop("r")
            theta[i] = np.arctan2(
                self.velocities[near, 1].mean(),
                self.velocities[near, 0].mean())
        theta += self.get_prop("nu")*(self.rndState.random(self.N)-0.5)
        active = self.get_prop("v")*np.column_stack((np.cos(theta),
     np.sin(theta)))
        return self.positions + (active + self.get_fluid_drift())*dt
```

## Three model ingredients

1. average neighbor velocity vectors within radius `r`;
2. add angular noise of width `nu`;
3. add self-propulsion speed `v` and local fluid drift.

## Periodic metric

The minimum-image displacement is essential: agents near opposite faces can still be neighbors. Note that the minimum-image wrap assumes periodic boundaries in both axes.

Condensed from the current repository Vicsek example (`examples/ex_vicsek_model_2d.py`); parameter names and hook are unchanged. Model: Vicsek et al. (1995).

# CAPSTONE: WHAT THE ALIGNMENT RULE ACTUALLY AVERAGES

In the notation of Vicsek et al. (1995), each agent $i$ updates its heading as

$$\theta_i^{n+1} = \arg\Big( \sum_{j:\,|\boldsymbol{x}_j - \boldsymbol{x}_i| < r} \boldsymbol{v}_j^{\,n} \Big) + \xi_i^n, \qquad \xi_i^n \sim \mathcal{U}\big[-\tfrac{\eta}{2}, \tfrac{\eta}{2}\big],$$

then moves with $\boldsymbol{x}_i^{n+1} = \boldsymbol{x}_i^n + \big[v\,(\cos\theta_i^{n+1}, \sin\theta_i^{n+1}) + \boldsymbol{u}(\boldsymbol{x}_i^n, t^n)\big]\Delta t$. Here $\eta$ is `nu`, $v$ is `v`, and $r$ is `r`.

- Because `move` resets `velocities` to the realized displacement over `dt`, the neighbor average uses ground-frame velocities (swimming plus fluid drift), not swimming headings alone. In a strong flow the fluid can dominate the "consensus".
- An agent always counts itself as a neighbor (distance $0 < r$), so the average is never empty.
- The condensed listing wraps both axes unconditionally; the repository version wraps only the axes whose `envir.bndry` is `periodic`, so use that form in a channel with walls.
- The noise is drawn once per step, so the statistics depend on $\Delta t$; this is why the capstone asks for at least two smaller $\Delta t$.
- A standard summary is the polarization $\varphi = \big|\sum_i \boldsymbol{s}_i\big|/(Nv) \in [0, 1]$, with $\boldsymbol{s}_i$ the self-propulsion velocity; $\varphi \approx 1$ is an aligned flock.

Vicsek et al. (1995), Eqs. (1)–(2) and their order parameter; Planktos example `ex_vicsek_model_2d.py`, commit `a1c2128`.

# RUN THE CAPSTONE, THEN MAKE THE RESULT REPRODUCIBLE

```
s = Vicsek(
    swarm_size=N, envir=env, init=IC, seed=2026)

collision_events = 0
for _ in range(72):        # 18 s at dt=0.25 s
    s.move(0.25, silent=True)
    collision_events += np.count_nonzero(
        s.ib_collision_idx >= 0)

print("agent-step collisions:", collision_events)

s.plot_all(
    movie_filename="vicsek_null.mp4",
    fps=4, fluid="vort")
```

## Capstone comparisons

Repeat with a passive swarm, multiple `nu/r` values, and at least two smaller $dt$. Compare polarization, retention, agent-step collisions, and downstream spread.

## Save with every result

- Planktos version/commit and Python/package environment;
- flow and mesh provenance plus checksums;
- both random seeds and all model parameters;
- time step, duration, collision mode, and active-agent count;
- code, raw histories, and plotting settings.

## Failure-state diagnostic

If a collision step fails, current Planktos marks `env.time=None`. Inspect the partial state; then restore consistently from histories or restart from a checkpoint.

# RESOURCES AND SOURCE TRAIL

## Start with the current software

- Planktos repository and README;
- repository `docs/` and runnable `examples/`;
- `examples/ib2d_data/README.txt` for the external IB2d dataset;
- this tutorial was audited against v1.0.3, commit `a1c2128` (2026-08-19).

> Record the exact software commit with every scientific result: current interfaces can differ from those shown in the 2022 paper.

## Primary scientific sources

Strickland et al. (2022), *Bulletin of Mathematical Biology* 84:72

Software architecture, agent models, collision handling, visualization, FTLE, and examples.

Hamlet et al. (2023), *Journal of Experimental Biology* 226, jeb244520

Gorgonian multiscale flow and capture application; article and reproduced figures are CC BY 4.0.

# REFERENCES


- W. C. Strickland, N. A. Battista, C. L. Hamlet, and L. A. Miller, “Planktos: an agent-based modeling framework for small organism movement and dispersal in a fluid environment with immersed structures,” *Bull. Math. Biol.* 84 (2022), 72, doi:10.1007/s11538-022-01027-1. Figs. 4, 5, 9 reproduced for education (article not openly licensed); FTLE qualifications from Sections 2.1 and 4.2.
- C. L. Hamlet, W. C. Strickland, N. Battista, and L. A. Miller, “Multiscale flow between the branches and polyps of gorgonians,” *J. Exp. Biol.* 226(5) (2023), jeb244520, doi:10.1242/jeb.244520. CC BY 4.0; Figs. 1 and 6A reproduced.
- W. C. Strickland *et al.*, *Planktos* agent-based modeling framework, v1.0.3, commit `a1c2128` (2026-08-19), GPLv3, https://github.com/mountaindust/Planktos; documentation https://planktos.readthedocs.io. Source of every code pattern (`README.md`, `setup.cfg`, `examples/`, docstrings) and of the `docs/_static` images.
- S. Kupfner Johnson and P. Hallock, “A review of symbiotic gorgonian research in the western Atlantic and Caribbean with recommendations for future work,” *Coral Reefs* 39 (2020), 239–258, doi:10.1007/s00338-020-01891-0. Adapted in Hamlet et al. (2023), Fig. 1.
- M. R. Maxey and J. J. Riley, “Equation of motion for a small rigid sphere in a nonuniform flow,” *Phys. Fluids* 26 (1983), 883–889, doi:10.1063/1.864230.
- G. Haller and T. Sapsis, “Where do inertial particles go in fluid flows?” *Physica D* 237 (2008), 573–583, doi:10.1016/j.physd.2007.09.027. Linearized form used by `inertial_particles`.

# REFERENCES (CONTINUED)


- T. Vicsek, A. Czirók, E. Ben-Jacob, I. Cohen, and O. Shochet, “Novel type of phase transition in a system of self-driven particles,” *Phys. Rev. Lett.* 75 (1995), 1226–1229, doi:10.1103/PhysRevLett.75.1226.
- S. C. Shadden, F. Lekien, and J. E. Marsden, “Definition and properties of Lagrangian coherent structures from finite-time Lyapunov exponents in two-dimensional aperiodic flows,” *Physica D* 212 (2005), 271–304, doi:10.1016/j.physd.2005.10.007.
- G. Haller, “Lagrangian coherent structures,” *Annu. Rev. Fluid Mech.* 47 (2015), 137–162, doi:10.1146/annurev-fluid-010313-141322.
- P. E. Kloeden and E. Platen, *Numerical Solution of Stochastic Differential Equations*. Springer, 1992, doi:10.1007/978-3-662-12616-5. Euler–Maruyama (§10.2) and Milstein (§10.3) schemes.
- J. R. Dormand and P. J. Prince, “A family of embedded Runge–Kutta formulae,” *J. Comput. Appl. Math.* 6 (1980), 19–26, doi:10.1016/0771-050X(80)90013-3. The RK45 pair behind `motion.RK45`.
- H. C. Brinkman, “A calculation of the viscous force exerted by a flowing fluid on a dense swarm of particles,” *Appl. Sci. Res.* A1 (1949), 27–34, doi:10.1007/BF02120313.

# REFERENCES (CONTINUED)


- W. C. Strickland, L. A. Miller, A. Santhanakrishnan, C. Hamlet, N. A. Battista, and V. Pasour, "Three-dimensional low Reynolds number flows near biological filtering and protective layers," *Fluids* 2 (2017), 62, doi:10.3390/fluids2040062. Analytic Brinkman profile behind `set_brinkman_flow`.
- D. W. Scott, *Multivariate Density Estimation*. Wiley, 1992, doi:10.1002/9780470316849. Scott's KDE bandwidth factor (`dist="density"`).
- N. A. Battista, W. C. Strickland, and L. A. Miller, "IB2d: a Python and MATLAB implementation of the immersed boundary method," *Bioinspir. Biomim.* 12 (2017), 036003, doi:10.1088/1748-3190/aa5e08. Source of the IB2d flow and mesh formats.
- FFmpeg project, https://ffmpeg.org. External movie writer used by Matplotlib for `plot_all(movie_filename=...)`.


> The workflow, object-model, Brinkman-layer, shared-clock, and FTLE-validity diagrams were drawn for these notes (TikZ). Code fragments are adapted from the Planktos repository (GPLv3) as credited on each slide.